Configuration Externalization Pattern

Category: Operations **One-liner:** Move configuration out of application code and deployment packages into a centralized store that services fetch at **runtime** (while they are running) — so changing config doesn't require rebuilding and **redeploying** (releasing a new build to the servers).

Part 0 — Concepts you need first

What is "configuration"?

Configuration is the set of values that control how an application behaves, but which **aren't the application's logic**. Typical examples:

- Database **connection strings** (the address and options used to connect to a database) and **credentials** (usernames and passwords)
- **API keys** (secret tokens identifying your program to another service) and **endpoint** URLs (the web addresses a service answers on) for external services
- **Timeout** durations (how long to wait before giving up), retry counts, **thread pool** sizes (how many worker threads a service keeps for handling requests)
- Feature flags — switches turning functionality on and off
- Region-specific values: currency, tax rate, language

The test for whether something is configuration: **would you want a different value in development versus production, or in one region versus another, without changing the code?** If yes, it's configuration.

What does "hardcoded" mean?

A value written directly into the source code:

```
String dbUrl = "jdbc:postgresql://prod-db-01:5432/orders";
```

Changing it means editing the source, **recompiling** (turning source code into a runnable program again), and redeploying. The value is **frozen into the built artifact** (the packaged output of the build).

What is a "deployment package"?

The bundle you actually ship — a **JAR** (Java ARchive, the packaged form of a Java program), a **container image** (a snapshot of an app plus everything it needs, used to start containers), a zip. "Bundled configuration" means the config file travels *inside* that package. Better than hardcoding (you can read it without recompiling), but **still frozen at build time**: to change it you rebuild the package and redeploy.

What is a "deployment cycle"?

The full process of getting a change into production: build → test → review → deploy → verify. In a well-run organisation this might take an hour. In a regulated one it might take days.

The core complaint of this chapter: that entire process, for changing one number.

What is an environment?

A separate copy of your system for a purpose — **development** (engineers experimenting), **staging/testing** (rehearsal), **production** (real users — the live system). Same code, different configuration: different databases, different API keys, different scale.

What is a secret?

Configuration that must not be disclosed — database passwords, API keys, **encryption keys** (the secret values used to scramble and unscramble data). **Secrets are configuration, but with much stricter handling requirements**, which becomes a recurring theme below.

Part 1 — The framing

Picture yourself as an architect designing a city. Would you place the blueprint of every building **inside** each one, hard-coded into the walls? Or keep it separate, accessible, and modifiable, so each building can adapt to changing needs?

The answer is obvious for physical structures. The same principle applies to software.

In **microservices** (an architecture of many small, independently deployed services), configuration determines how efficiently applications run and adapt. Whether it's network settings, database parameters, or feature toggles, **each microservice has a set of configurations governing its behaviour**.

Traditionally these are hardcoded into the application or bundled within the deployment package. But is that efficient? **What happens when you need to change a parameter? A new deployment cycle for a minor tweak?**

Part 2 — The problem

Volume and diversity

Picture your architecture as a bustling city, each microservice a building with its own purpose and configuration. Now imagine **each building is from a different era, built with diverse architectural styles and materials.** That's what a microservices architecture often looks like — many services, possibly written in **different languages using different technology stacks** (each its own set of languages, frameworks and libraries).

Managing configuration across all of them quickly becomes a nightmare, because each stack has its own conventions for where config lives and how it's read.

The drawbacks of hardcoding

Hardcoding is **like permanently etching the blueprint onto a building's facade**. It seems fine initially — everything's in one place. The downsides appear when you need to change something.

Hardcoding ties configuration **tightly to the application code**. Consequences:

- **Any change, no matter how minor, requires a new version of the application followed by a complete redeployment**
- This **slows the adaptability** of your microservices
- **It introduces risk.** If a configuration error slips into production, you face **another round of building and deployment** — wasting resources and, in the worst case, causing **significant downtime** (time when the system is unavailable to users)

> That last point is the one worth dwelling on. A typo in a database **hostname** (the network name of a machine) is a five-second fix conceptually, but a full deployment cycle in practice — and you're doing it while production is broken.

The burden of scale

These issues are magnified in microservices by the sheer scale of the system.

> Return to the city analogy: now imagine you need to modify the blueprint of **every single building**. That's what engineers face when configuration is hardcoded or bundled within each microservice.

In a **monolithic** architecture (one application deployed as a single unit), configuration is usually **centralized and managed in one place**. In **microservices**, each service comes with its own configuration maintained independently. This lack of centralization **amplifies the workload** and **increases the risk of inconsistencies and errors** across the system.

> A concrete inconsistency: you **rotate** a database password (replace it with a new one, as routine security hygiene) and update it in fourteen services. You miss one. That service keeps working until its **connection pool** (its set of already-open database connections, reused across requests) recycles — possibly hours later — and then fails in a way that looks unrelated to anything you did.

Inflexibility of deployments

Microservices are known for adaptability — different services deployed independently, enabling faster releases and efficient **scaling** (adding capacity to handle more load). **Traditional configuration management severely hampers this flexibility.**

With hardcoded or bundled configuration, **each minor change could necessitate a full redeployment of a service**. This is inefficient, and it **negates the independent deployment advantage** that is microservices' main selling point. It also leads to **inconsistencies in configuration across environments** — development, testing, production.

Inability to adapt to dynamic environments

Microservices often operate in dynamic environments where conditions change rapidly, and those conditions influence what configuration gives optimal performance:

- **Network settings** may need adjusting based on **traffic patterns**
- **Feature toggles** may need flipping based on **user behaviour**

With traditional configuration management, **responding to these conditions in real time (as they happen) becomes nearly impossible.** Your services are **stuck with outdated configuration until the next deployment cycle**, impacting performance and user experience.

Security implications

Hardcoded configuration can lead to **sensitive information — database credentials, API keys — being exposed or leaked**. This is a security risk with serious consequences for your system and your users' data.

> The most common way this happens in practice: a credential hardcoded in source is **committed** to **version control** (a system such as **Git** that records every change to files, with who made it and when), and now it exists in the **repository**'s (the project's stored files and history) history **forever**, visible to everyone with read access — and permanently, since rewriting Git history is painful and rarely complete. Rotating the credential is the only real remedy.

Part 3 — The solution

Apply **separation of concerns** (keeping different responsibilities in different places) to configuration management: **externalize configuration from application code**, storing and managing it in a **separate, centralized location**.

> Take all the blueprints of the city's buildings and keep them in a **central town hall**, away from the buildings themselves. The blueprints become easy to update and review without disturbing the buildings.

Mechanically: configuration is no longer part of the deployment package. It's stored externally in a **configuration server** (a service whose only job is to hand configuration to other services) or a file in a network-accessible location. The microservice **retrieves configuration at runtime**, ensuring it always has the most up-to-date values.

Why this is advantageous

A unified, centralized location for managing all configuration. This directly addresses the volume and diversity problem: **regardless of a microservice's language or technology stack, configuration is retrieved from the same place**, simplifying management and reducing errors.

Support for dynamic environments. Since configuration is retrieved at runtime it can be **updated in real time** as conditions change. Services **no longer wait for the next deployment cycle** to adapt — they're as dynamic as the environment they operate in.

Preserved independent deployment. Separating configuration from code upholds the microservices principle of independent deployment. Configuration changes **no longer necessitate a full redeployment**; they're updated centrally and become immediately available.

Enhanced security. With configuration externalized and centrally managed, it's **easier to enforce security policies and controls**. Sensitive information is **no longer scattered across microservices** — it's securely stored and managed in one place, reducing the risk of exposure or leaks.

Efficient scalability. The pattern scales with your architecture. As new microservices are added, their configuration is simply included in the centralized location. **No need to navigate through each service** to update configuration.

Consistency across environments. Moving from development to testing to production can feel like entering different countries with different rules. Configuration for all environments is **centrally managed**, keeping them in sync and reducing errors when promoting between them.

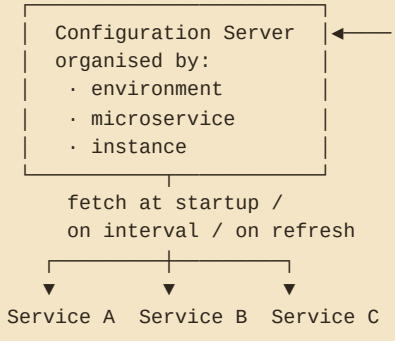
Part 4 — Architecture

At the heart of the pattern are two components: **the microservice** and the **configuration server** (or configuration file).

The interaction

The microservice, **at startup or at pre-defined intervals**, sends a request to the configuration server asking for its configuration. This can include **database connection settings, API keys, or any other environment-specific data**. The server responds with what that service needs.

What's notable is that this exchange happens **dynamically, at runtime**. The microservice can adapt in real time to new configuration **without needing a restart or redeployment**, reducing downtime and enhancing **agility** (the ability to change quickly).



The configuration server

The **nerve centre** — think of it as the command centre overseeing and controlling all configuration. It stores, retrieves, and manages configuration for all microservices.

To streamline things, configuration can be organised:

- **By environment** — development, staging, production
- **By microservice**
- **By instance** of a microservice (one particular running copy)

> Like a well-organised library, with each configuration neatly categorised and easy to find. This organisation is what mitigates the challenges of traditional configuration management — it's not just centralized, it's *navigable*.

Dynamic adaptability in practice

Suppose you have a microservice interacting with a database, with connection parameters stored as configuration. Suddenly the database connection parameters change.

Instead of redeploying the microservice with the new parameters, you update the configuration server. The microservice, when it next retrieves configuration, **receives the updated parameters and continues its operation seamlessly.**

Security — safeguarding sensitive configuration

Returning to the library analogy: **not all books are open for everyone to read.** Some configuration contains sensitive data needing protection.

The configuration server can secure sensitive configuration through **encryption** (scrambling data so only key-holders can read it) or **access controls** (rules about who may read or change what). By centralizing management of these values, it's **easier to enforce strict security policies**, reducing the risk of inadvertent exposure or malicious attack.

Scalability and consistency

As you add more microservices, you simply add their configuration to the server — **no worrying about individually managing configuration for each new service.**

And by externalizing configuration you maintain **consistency across environments**. You **switch configuration as you move through environments** — no environment-specific code, no configurations getting mixed up.

Part 5 — Worked example: Spring Cloud Config

Spring Cloud Config (part of Spring Cloud, the microservices toolkit built on the Spring Java framework) is widely adopted in the Java community for centralized configuration management in microservices, due to its simplicity and efficiency.

The configuration server

```
@SpringBootApplication
@EnableConfigServer
public class ConfigServer {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServer.class, args);
    }
}
```

With just these few lines you've turned an application into a configuration server. The `@EnableConfigServer` **annotation** (a label starting with `@` that tells the framework to add behaviour) does the work.

The configuration store — a Git repository

Now tell the server where to find the configuration, in a **properties file** (a plain-text file of `key=value` settings):

```
spring.cloud.config.server.git.uri=https://github.com/yourusername/config-repo
```

This tells the configuration server to fetch configuration from the specified Git repository.

> **Using Git here is genuinely clever, and worth understanding rather than skimming.** By storing configuration in a version-controlled repository you get, for free: a full history of every change and who made it, **diffs** showing exactly what changed, **pull requests** (proposed changes that others review before they are merged) and **code review** *for configuration*, and the ability to **roll back to any previous state** with a standard Git operation. Configuration becomes a first-class engineering artifact rather than a value someone typed into a form.

The client setup

In the microservice's **build file** (the file, such as `pom.xml` or `build.gradle`, that lists what the project needs), add the Spring Cloud Config Client **dependency** (an external library the project uses). Then annotate the main class with `@EnableConfigClient`. The microservice is now a configuration client, ready to consume configuration from the server.

Pulling configuration — the `@Value` annotation

```
@Value("${my.property}")
private String myProperty;
```

`my.property` is a property defined in the configuration store. `@Value` fetches its value and assigns it to the field. **Note how little the application code knows** — it references a property name, not a location or a mechanism.

Refreshing without restarting

```
POST /actuator/refresh
```

Sending a **POST** request (the HTTP method for submitting an action) to this endpoint — provided by **Spring Boot Actuator**, Spring's built-in management module — triggers the configuration client to **re-fetch configuration from the server** — new configuration applied **without restarting the microservice.**

This is the payoff of the whole pattern in one endpoint: change a value centrally, hit refresh, done.

Encryption and decryption

```
encrypt.key=MySuperSecretKey
```

This sets the encryption key used to encrypt and decrypt properties. To encrypt a property use the `/encrypt` endpoint; to decrypt, `/decrypt`.

> ⚠️ **Notice the irony, because it matters in practice.** `encrypt.key` is itself a secret, sitting in a plain properties file. You've encrypted all your secrets and left the key to them in the open. In production this master key belongs in a dedicated **secrets manager** (a service built to store, hand out and rotate secrets securely) — **HashiCorp Vault, AWS Secrets Manager, or a cloud KMS** (key management service — a cloud service that stores encryption keys in protected hardware) — with the config server retrieving it at startup. **The pattern relocates the secret problem; it doesn't eliminate it.** For secrets specifically, a purpose-built secrets manager usually beats a general configuration server.

Part 6 — Considerations and implications

Data security

While the pattern aids security, **it also introduces a new surface area for potential attacks** (another place an attacker could target). You are now storing sensitive configuration data **remotely**. How secure is your storage? Is your data transfer secure?

To ensure security, consider **encryption**. The `MySuperSecretKey` example is just the tip of the iceberg — from using **secure and encrypted connections** to **regularly rotating keys**, you need to stay ahead in the security game.

The configuration server as a single point of failure

While centralizing simplifies management, it **introduces a single point of failure** (one component whose failure stops everything).

> **What happens if your configuration server goes down? All your microservices would be paralysed, unable to fetch or refresh their configuration.**

Mitigation: introduce **redundancy** (spare copies). Run **multiple instances** of your configuration server in a **high-availability configuration** (set up so the service stays up even if one copy fails). And **regular health checks** (automatic "are you alive?" probes) **are crucial.** See System Design Fundamentals for these ideas in general.

> Add to this the mitigation the course covers under performance: **client-side caching of last-known-good configuration** (each service keeps a local copy of the last values it fetched successfully). This is what makes the whole trade acceptable. You've swapped "config changes require a redeploy" for "config server outage could take everything down" — and caching converts that second risk from fatal to merely inconvenient, because running services keep using the values they already have. This is the identical **resilience** trick (a way of staying up when parts fail) used in Service Discovery.

Configuration versioning and rollbacks

The pattern allows easier scalability and quicker feature rollouts. But **what happens when you need to roll back?** Can you easily revert to a previous configuration if something goes wrong?

The good news: version control systems like **Git** provide the solution. Storing configuration in a version-controlled repository means you can **easily roll back to a previous state**.

But this **requires good configuration management practices**: Are your changes well documented? Are versions correctly labelled? Git gives you the capability; discipline is what makes it usable at 3am during an **incident** (a production outage or emergency).

Service discovery and the config service

In a dynamic microservices environment, services come up and down and scale based on **load** (the amount of work arriving). **How do new services discover the config service?**

Hardcoding the configuration server's address in each microservice isn't a scalable solution — and note the irony: you'd be hardcoding a location in order to avoid hardcoding configuration.

This is where **service discovery** (a registry that tells services where other services currently are) comes in. Tools like **Netflix's Eureka** or **HashiCorp's Consul** (two popular service registries) dynamically discover services in your architecture, **including the configuration server**. *(See Service Discovery.)*

Performance implications

Fetching configuration remotely may introduce latency (extra delay), especially if the configuration server is under heavy load or network issues arise.

Caching helps reduce this latency — Spring Cloud Config clients **cache remote properties by default**.

But caching brings its own considerations:

- **How often do you refresh the cache?** Too often and you hammer the server; too rarely and changes take a long time to take effect
- **What's the impact on memory usage?**

> It's a balancing act between **performance and freshness of configuration.**

Part 7 — Real-world use cases

E-commerce platform

Several microservices — Inventory, Order, User Management — each interfacing with various databases and APIs, each with its own configuration. With **hundreds of services running across production, staging, and development**, managing configuration is a nightmare.

The pattern externalizes all configuration to a central server. Each service fetches what it needs. **Changes are made in one place and reflected across the system.**

Financial services firm

A highly regulated environment. The firm must ensure **sensitive data like API keys and database passwords are not exposed**.

Using this pattern, all sensitive configuration is stored in the central configuration server, **possibly in encrypted form**. Services fetch configuration **over a secure channel** (an encrypted connection) and use it. **No sensitive data is stored in the service codebase or containers** (the **codebase** is the source code; **containers** are the packaged, isolated running apps) — which is exactly what **auditors** (people who check a company follows its rules and regulations) want to see.

Global fast-food chain

Outlets in numerous countries, each with an application managing orders, inventory, and logistics. But **each country has its own tax laws, currency, and language preferences.**

The chain uses the pattern to manage **country-specific configuration**. When an application boots up it **fetches the configuration for the country it operates in** — like having a personalized configuration per country, from one shared codebase.

Tech startup

A startup developing a streaming service, following **agile** practices (working in short cycles and shipping small changes often) with **multiple deployments each day**. Each deployment may require configuration changes — **feature flags or A/B testing parameters** (A/B testing shows two versions to different users to compare which works better).

The pattern lets them easily manage this dynamic configuration. Each service fetches updated configuration whenever it needs it, making **deployments faster and smoother**.

Telecommunications giant

Complex network infrastructure managed by numerous microservices. **Network parameters like bandwidth** (data-transfer capacity), **routing information, and security protocols** (agreed rules for securing communication) need meticulous management.

All network configuration is stored centrally; each microservice fetches what it needs, giving efficient management of complex configuration.

Part 8 — Common beginner misconceptions

"Externalizing config means putting it in a file next to the app." That's still bundled config. Externalizing means it lives **outside the deployment artifact**, retrieved over the network.

"A config server is the right place for secrets." It's *better* than hardcoding, but a dedicated secrets manager (Vault, AWS Secrets Manager) gives you **leasing** (secrets that are handed out for a limited time

and then expire), automatic rotation, fine-grained access policies, and **audit logs** (records of who accessed which secret, when) that a general config server doesn't.

"Environment variables are the same thing." (An **environment variable** is a named value the operating system passes to a program when it starts.) They're a good, simple form of externalization and often sufficient. What they don't give you is **central management, versioning, or refresh without a restart** — changing an environment variable means restarting the process.

"If the config server is down, everything stops." Only without client-side caching. With caching, **running services keep running**; you just can't start new ones or push changes. Design for that.

"Config changes are low risk because there's no deploy." The opposite concern applies — no deploy means **no build, no tests, no review** unless you deliberately add them. That's precisely why backing the store with Git and requiring pull requests matters.

"One config server for everything." Consider separating by **trust boundary** (a line between parts of the system that should not automatically trust each other). Not every service should be able to read every other service's configuration.

Part 9 — Check your understanding

1. What's the difference between hardcoded, bundled, and externalized configuration?
2. Why does hardcoded configuration undermine the main benefit of microservices?
3. What does backing the config store with Git give you for free?
4. What's the central irony of the `encrypt.key` property?
5. Why do you need service discovery in order to do configuration externalization well?
6. What makes the config server's single-point-of-failure risk acceptable?

*(Answers: 1 — hardcoded is in source and needs recompiling; bundled is in the deployment package and needs rebuilding and redeploying; externalized lives outside the artifact and is fetched at runtime; 2 — independent deployability, because a one-line config change forces a full redeploy of the service; 3 — change history, authorship, diffs, code review, and rollback to any previous state; 4 — it's a secret protecting all other secrets, stored in **plaintext** (unencrypted), so it belongs in a real secrets manager; 5 — hardcoding the config server's address in every service would reintroduce the exact problem you're solving; 6 — client-side caching of last-known-good values, so running services survive a config server outage.)*

Part 10 — Interview soundbite (30 seconds)

> "Configuration externalization moves config out of application code and deployment packages into a central config server that services fetch at runtime. The problem it solves is that hardcoded config ties every change to a full build-and-deploy cycle, which destroys the independent deployability that's the whole point of microservices — and means you can't react to changing conditions until the next release. It also scatters secrets like database credentials across every service and into version control history. With a config server you organise config by environment, service, and instance; services fetch at startup or on an interval, and Spring Cloud Config exposes an actuator refresh endpoint so you can apply changes without restarting. It's typically backed by a Git repo, which is the clever part — you get versioning, diffs, code review, and rollback on your configuration for free. The main trade-offs are that the config server becomes a single point of failure, so it needs HA plus client-side caching of last-known-good values so a config outage degrades rather than kills you; you've created a new attack surface holding all your secrets, so encryption in transit and key rotation matter; and services need service discovery to find the config server, because hardcoding its address would reintroduce the exact problem you're solving. For secrets specifically I'd use a dedicated secrets manager like Vault rather than a general config server, since you get leasing, rotation, and audit logging."

Part 11 — Implementation

A. Runnable demo

Full source: `demos/ConfigExternalizationDemo.java`

```
cd demos && ./gradlew runCh13
```

Layered resolution — later layers override earlier ones:

```
Map<String, String> fetch() {
    Map<String, String> merged = new LinkedHashMap<>();
    layers.values().forEach(merged::putAll);   // default <- environment <- instance
    return merged;
}
```

The cache that survives a server outage:

```
void refresh() {
    try {
        cache = server.fetch();
        cachedVersion = server.version();
    } catch (IllegalStateException e) {
        // server unreachable - KEEP the previous config rather than failing
    }
}
```

And the detail that makes runtime flips work — read config **at use time**, not at startup:

```
String checkout() {
    return config.flagEnabled("feature.newCheckout")   // read on every call
            ? "NEW checkout flow" : "legacy checkout flow";
}
```

B. What you should see

```
=== 3. Change a value + refresh, NO restart ===
  before refresh: ... timeout=1000ms ...
  after refresh:  ... timeout=5000ms ...
  >> same running process, new value

=== 4. Layering: instance overrides environment overrides default ===
  ... timeout=9000ms ...
  >> instance layer won (9000), overriding environment (5000)

=== 5. Feature flag flipped centrally ===
  checkout now: legacy checkout flow
  checkout now: NEW checkout flow
  and svc2 too: NEW checkout flow
  >> every service flipped at once, with no deployment

=== 6. Config server outage ===
    [client] server DOWN - keeping cached config (v8)
  checkout still: NEW checkout flow
  >> running services keep working on cached config
```

Section 5 is the payoff — one central change flipped behaviour in every running service simultaneously, with no build and no deploy. **Section 6 is the safety net** that makes centralizing acceptable: the config server died and nothing stopped.

C. Production — Spring Cloud Config

The server:

```java
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {
    public static void main(String[] args) { SpringApplication.run(ConfigServerApplication.class, args); }
}
```

```yaml
server.port: 8888
spring:
  cloud.config.server.git:
    uri: https://github.com/acme/config-repo
    default-label: main
    clone-on-start: true
    search-paths: '{application}'          # per-service folders
  security.user:
    name: configuser
    password: ${CONFIG_SERVER_PASSWORD}    # itself injected, not committed
```

The repository layout — this is what gives you environment layering:

```
config-repo/
├── application.yml                # defaults for EVERY service
├── application-prod.yml           # defaults for every service in prod
├── order-service.yml              # all environments of one service
├── order-service-dev.yml
└── order-service-prod.yml         # most specific: wins
```

The client:

```yaml
spring:
  application.name: order-service          # picks which files apply
  config.import: "optional:configserver:http://config-server:8888"
  cloud.config:
    fail-fast: true                        # don't start with wrong config
    retry: { initial-interval: 1000, max-attempts: 6, multiplier: 1.5 }
```

Refresh without restart — the crucial bit is `@RefreshScope` (a **bean** is an object that Spring creates and manages; `curl` is a command-line tool for sending HTTP requests):

```java
@RefreshScope                              // bean is recreated on refresh
@Component
public class CheckoutService {

    @Value("${feature.newCheckout:false}")
    private boolean newCheckoutEnabled;

    public String checkout() {
        return newCheckoutEnabled ? newFlow() : legacyFlow();
    }
}
```

```
curl -X POST http://order-service:8080/actuator/refresh
```

For **fleet**-wide refresh (across every running instance), **Spring Cloud Bus** broadcasts over **Kafka** or **RabbitMQ** (two popular message brokers — servers that pass messages between programs) so one call refreshes every instance:

```
curl -X POST http://any-instance:8080/actuator/busrefresh
```

Secrets belong in a secrets manager, not here: (`KUBERNETES` authentication means Vault trusts the identity Kubernetes gives each **pod** — its unit of running containers.)

```
spring.cloud.vault:
  uri: https://vault:8200
  authentication: KUBERNETES            # pod identity, no static credentials
  kv: { enabled: true, backend: secret }
```

Vault gives you **leasing, automatic rotation, and audit logs** — none of which a general config server provides. Keep ordinary configuration in Git and secrets in Vault.

Kubernetes-native alternative — often all you need. (**Kubernetes** is the standard platform for running containers; a **ConfigMap** is its built-in object for storing configuration, which can be **mounted as a volume** — exposed to the container as files.)

```
apiVersion: v1
kind: ConfigMap
metadata: { name: order-service-config }
data:
  http.timeoutMs: "3000"
  feature.newCheckout: "true"
---
# Mounted as a volume, changes propagate to the file within ~1 min
volumes:
  - name: config
    configMap: { name: order-service-config }
```

Note: ConfigMaps mounted as **volumes** update live; ConfigMaps consumed as **environment variables** do **not** — the pod must restart. That trips people up constantly.

D. Implementation pitfalls

1. Reading config once into a `static final` field (a Java field set once and shared by the whole class). Then refresh does nothing, because the value was captured at **class-load time** (when Java first loaded the class into memory). Use `@RefreshScope`, or read through a provider at each use.

2. No client-side caching or `fail-fast` misconfigured. Without caching, a config server outage takes down every service. With `fail-fast: true` **and** retry configured, startup waits and retries rather than booting with defaults that are silently wrong.

3. Secrets in the Git config repo. Once committed they're in the history permanently. Use Vault or a **sealed-secrets** mechanism (secrets stored encrypted in Git that only the cluster can decrypt), and treat any leaked credential as needing rotation, not deletion.

4. The encryption key stored beside the encrypted values. The irony flagged in Part 5 — you've locked the door and left the key in it. The master key belongs in a KMS or in Vault, injected at startup.

5. No validation of config before it propagates. A typo in a Git commit reaches production in seconds with no build or test to catch it. Add a **CI** job (**continuous integration** — an automated check run on every change) that validates the config repo, and require pull requests — the speed of config changes is a hazard as well as a feature.

6. Changing config in production directly. If your repo is the **source of truth** (the one authoritative copy), an **out-of-band** change (one made outside the normal process) gets silently reverted by the next refresh, producing a genuinely baffling incident.

7. A single config server instance. It's a single point of failure for the whole estate. Run at least two **replicas** (identical running copies) behind a service, and monitor them.

8. Hardcoding the config server's address. Self-defeating. Use service discovery or a stable Kubernetes Service **DNS** name (DNS is the system that turns names into network addresses).

9. Assuming env-var ConfigMaps refresh. They don't. Only volume mounts propagate without a restart.

Related patterns

- **Service Discovery** — how services locate the config server; identical HA and caching concerns
- **Sidecar** — a sidecar can fetch config and expose it locally to the application

- **Strangler Fig** — the feature flags driving migration routing live in externalized config
- **Circuit Breaker** — breaker thresholds are configuration you'll want to tune without redeploying